

BCSE204L Design and Analysis of Algorithms

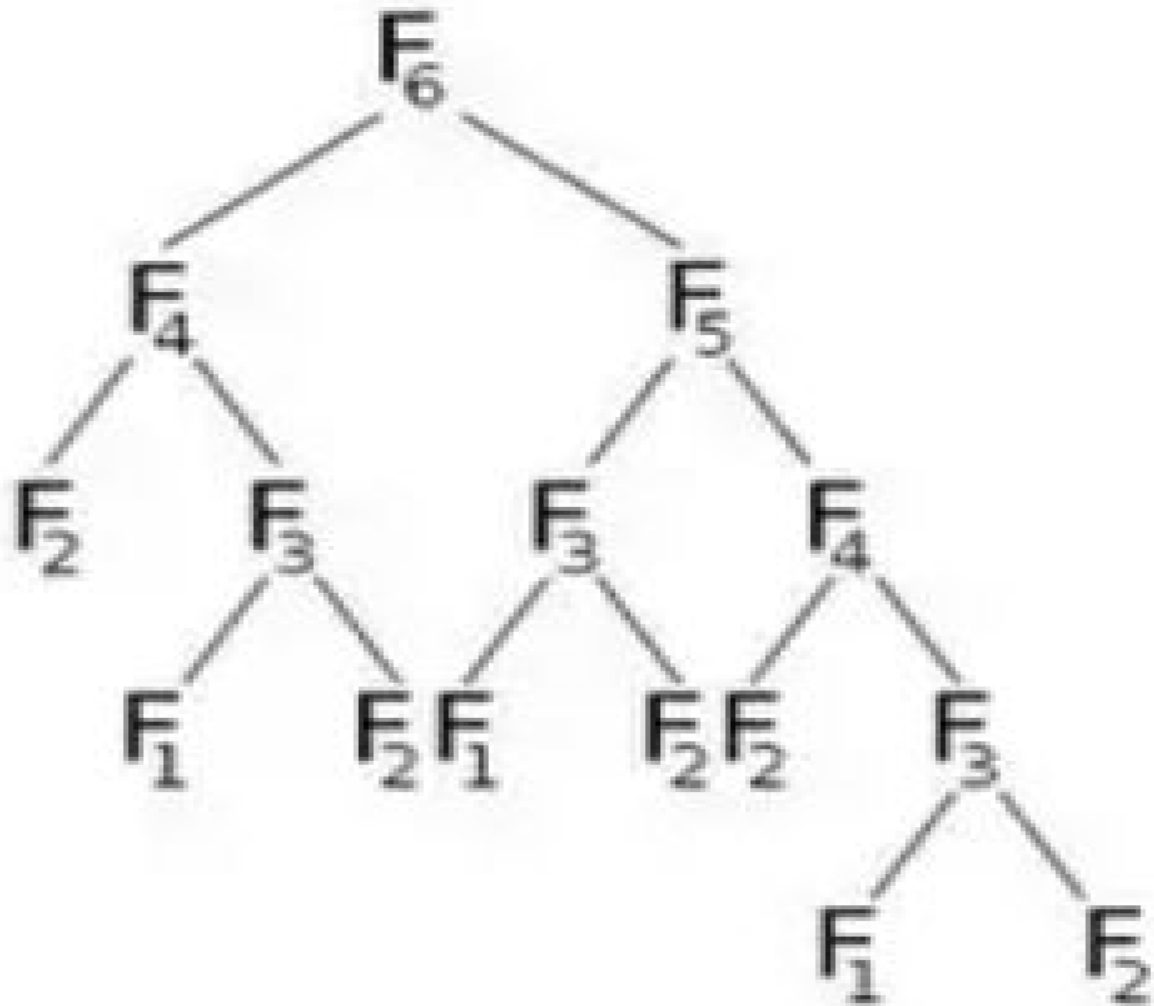
Module-2

Topic-1: Dynamic Programming

Divide-and-conquer Vs Dynamic programming

- Why did recursion work so well with divide-and-conquer?
 - A problem is expressed in terms of subproblems that are substantially smaller, say half the size.
 - Because of this drop in problem size, the full recursion tree has only logarithmic depth and a polynomial number of nodes
- When dynamic programming method enter the picture?
 - A problem is reduced to subproblems that are only slightly smaller — say $A[j]$ relies on $A[j-1]$.
 - Because of this, recursion tree generally has polynomial depth and an exponential number of nodes.

Need of dynamic programming

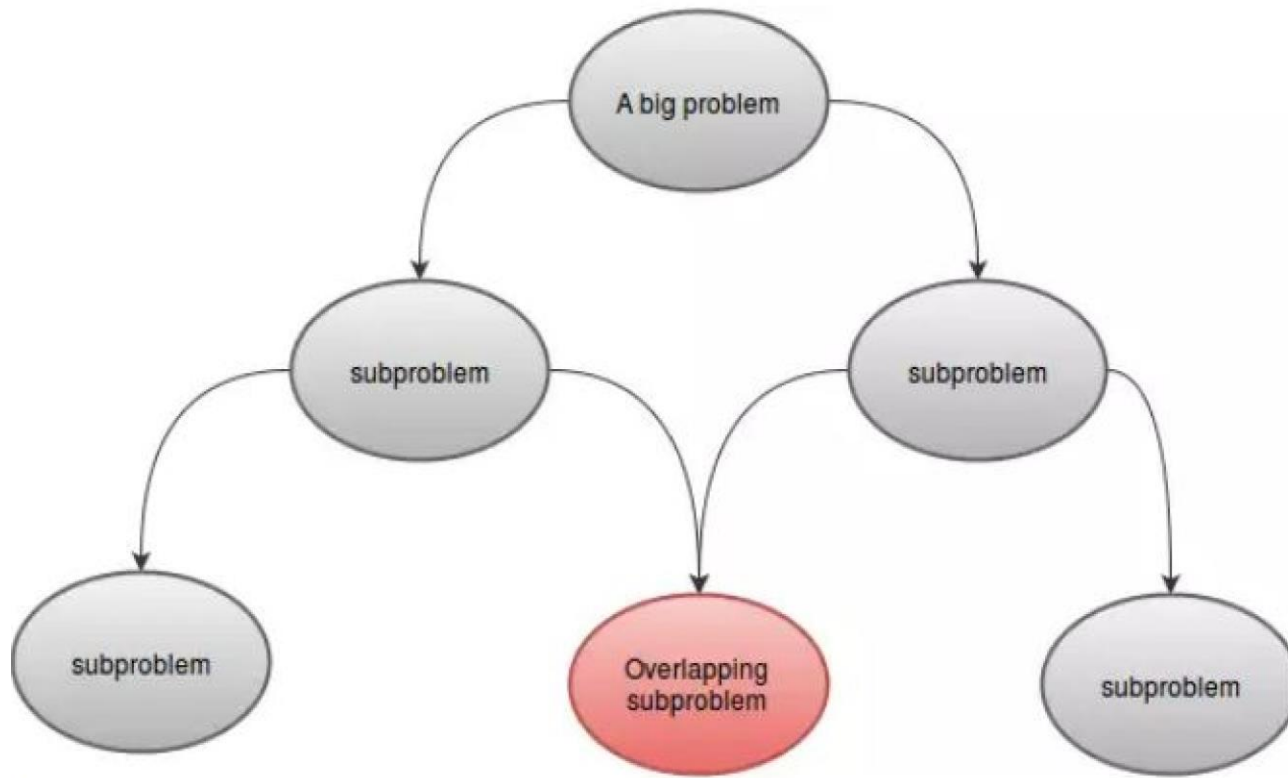


```
Procedure F(n) :  
  if n is equal to 1  
    Return 1  
  else if n is equal to 2  
    Return 1  
  end if  
  Return F(n-1) + F(n-2)
```



Simple
optimization
using
memoization:

```
Procedure F(n):  
  if n is equal to 1  
    Return 1  
  else if n is equal to 2  
    Return 1  
  else if dp[n] is not equal to -1  
    Return dp[n]  
  else  
    dp[n] = F(n-1) + F(n-2)  
    Return dp[n]  
end if
```



A problem can be solved using Dynamic Programming if it has some characteristics:

- 1. Subproblems**
- 2. Overlapping Subproblems**
- 3. Optimal Substructure**

Optimal Substructure in DP

- Key principle in DP
- A problem is said to satisfy the Principle of Optimality if the sub-solutions of an optimal solution of the problem are themselves optimal solutions for their subproblems
- Eg.:- The shortest path problem satisfies the Principle of Optimality.
- This is because if $a, x_1, x_2, \dots, x_n, b$ is a shortest path from node a to node b in a graph, then the portion of x_i to x_j on that path is a shortest path from x_i to x_j

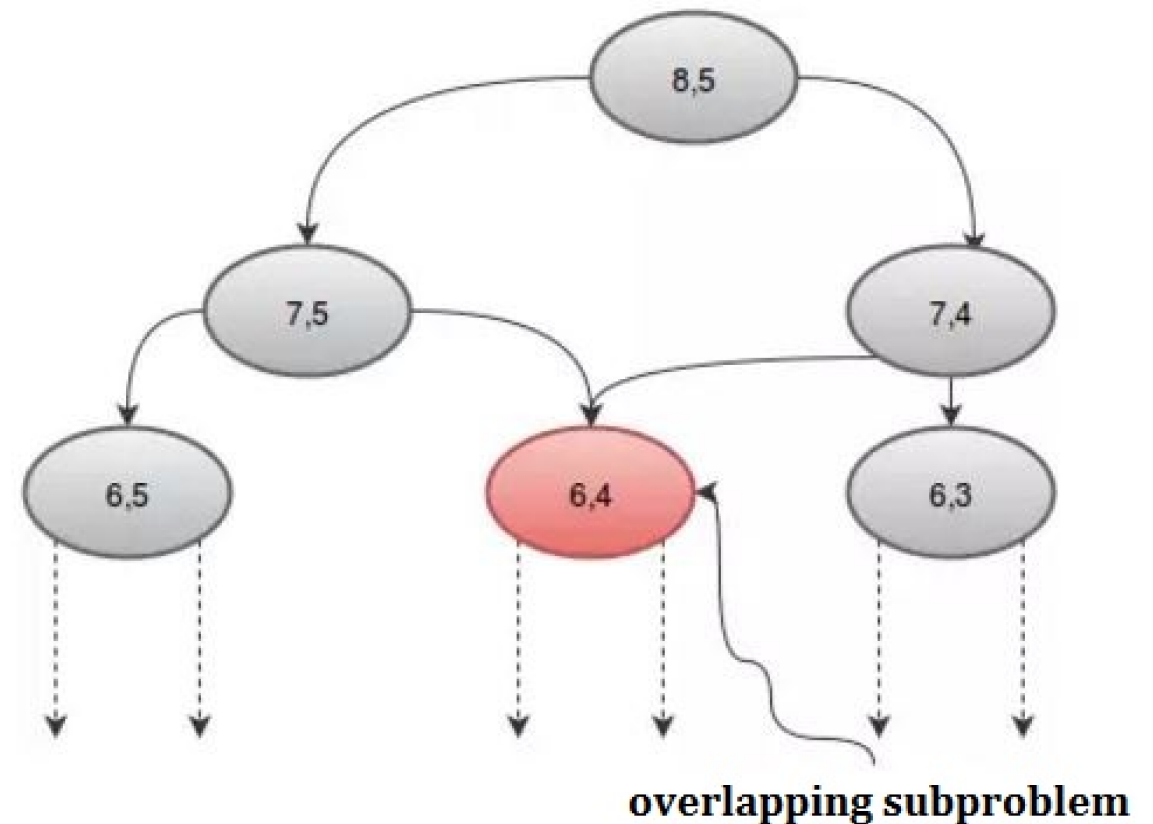
DP vs Greedy

- Both used to solve optimization problems
- Greedy follows predefined procedure
- D.P. tries out all possible solutions and finds the optimal one
- D. P. uses recursive formula
- D. P. follows principle of optimality. ie., solve problem by taking sequence of decisions to get optimal solution (in every step)
- In greedy one time decision making
- In DP optimal solutions to subproblems are retained so as to avoid recomputing their values
 - Memoization and tabulation

Example- From n items, in how many ways you can choose r items? Implement a program to do this.

$${}^nC_r = {}^{n-1}C_r + {}^{n-1}C_{r-1}$$

```
Procedure nCr(n, r):  
  if r is equal to 1  
    Return n  
  else if n is equal to r  
    Return 1  
  end if  
  Return nCr(n-1, r) + nCr(n-1, r-1)
```




```
Procedure nCr(n, r) :  
  if r is equal to 1  
    Return n  
  else if n is equal to r  
    Return 1  
  else if dp[n][r] is not equal to -1  
    Return dp[n][r]  
  end if  
  dp[n][r] := nCr(n-1, r) + nCr(n-1, r-1)  
  Return dp[n][r]
```

Memoization and Tabulation

Memoization: Top Down

```
int solve(int x)
{
    if (x==0)
        return 1;
    if (dp[x]!=-1)
        return dp[x];
    return (dp[x] = x * solve(x-1));
}
```

1. Memory is not filled in a sequential manner
2. Starts from the top most destination state and compute its answer , till reach the bottom-most base state

Tabulation: Bottom Up

```
int dp[0] = 1;
for (int i = 1; i<=n; i++)
{
    dp[i] = dp[i-1] * i;
}
```

1. Starts its transition from the bottom-most base case $dp[0]$ and reaches its destination state $dp[n]$.
2. DP table is being populated sequentially and we are directly accessing the calculated states from the table

	Tabulation	Memoization
State	State Transition relation is difficult to think	State transition relation is easy to think
Code	Code gets complicated when lot of conditions are required	Code is easy and less complicated
Speed	Fast, as we directly access previous states from the table	Slow due to lot of recursive calls and return statements
Subproblem solving	If all subproblems must be solved at least once, a bottom-up dynamic-programming algorithm usually outperforms a top-down memoized algorithm by a constant factor	If some subproblems in the subproblem space need not be solved at all, the memoized solution has the advantage of solving only those subproblems that are definitely required
Table Entries	In Tabulated version, starting from the first entry, all entries are filled one by one	Unlike the Tabulated version, all entries of the lookup table are not necessarily filled in Memoized version. The table is filled on demand.

Dynamic programming

- Dynamic programming solves problems by combining the solutions to subproblems
- Dynamic programming applies when the subproblems overlap - that is, when subproblems share subsubproblems
- A divide-and-conquer algorithm does more work than necessary, repeatedly solving the common subsubproblems.
- A dynamic-programming algorithm solves each subsubproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time it solves each subsubproblems

DP Examples

- 0/1 knapsack
- Floyd-Warshall algorithm (All pair shortest path)
- LCS (Longest Common Subsequence)
- TSP (Travelling Salesman Problem)
- Matrix chain multiplication
- Assembly Line Scheduling
-and many more